

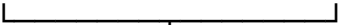
1

| MCP



| Xano Developer |

| MCP |



documentação /

validação



| VS Code |

| |

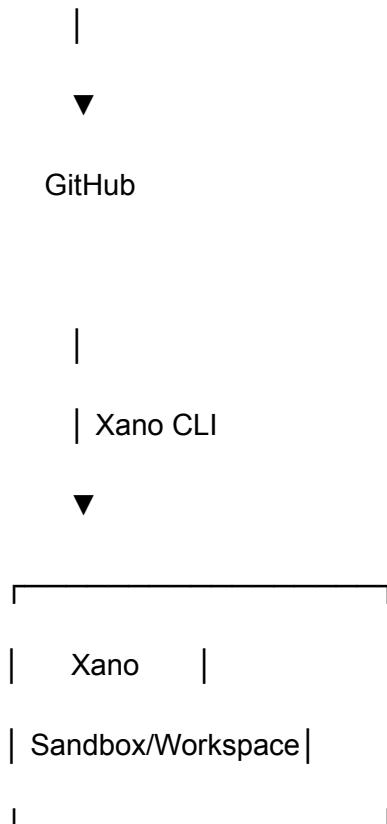
| XanoScript Language Server |

| |

| .xs |



Git



O **Xano Developer MCP** é destinado ao desenvolvimento com XanoScript: ele fornece documentação, validação e referências para o agente de IA. Ele não gerencia diretamente a instância Xano. A comunicação com o workspace Xano é feita separadamente pelo **Xano CLI**.

O aluno deverá possuir:

O Xano Developer MCP atualmente requer **Node.js 18 ou superior**.

3. Instalação do Visual Studio Code

Baixe e instale o Visual Studio Code pelo site oficial:

[Visual Studio Code](#)

Durante a instalação, recomenda-se habilitar as opções de integração com o Windows, especialmente a opção para adicionar o VS Code ao PATH.

Depois da instalação, abra o VS Code.

4. Instalação do Git

Baixe o Git pelo site oficial:

[Git](#)

Após a instalação, abra o PowerShell e execute:

```
git --version
```

Se a instalação estiver correta, será apresentada a versão instalada.

Também é recomendável configurar o nome e o e-mail utilizados nos commits:

```
git config --global user.name "Seu Nome"
```

```
git config --global user.email "seu-email@example.com"
```

Verifique:

```
git config --global --list
```

5. Instalação do Node.js

Baixe a versão LTS do Node.js:

[Node.js](#)

Após instalar, abra um novo terminal e execute:

```
node --version
```

```
npm --version
```

O comando `node --version` deve retornar uma versão igual ou superior à 18.

O Node.js será utilizado pelo Xano CLI e pelo Xano Developer MCP.

6. Instalação da extensão XanoScript Language Server

No VS Code:

1. Abra **Extensions**;
2. Pesquise por **XanoScript Language Server**;
3. Instale a extensão oficial.

O Language Server é responsável por fornecer suporte à linguagem XanoScript dentro do editor, como:

- syntax highlighting;
- autocomplete;
- diagnóstico de erros;
- validação;
- assistência durante a edição dos arquivos `.xs`.

Atenção

Não confundir:

XanoScript Language Server

com a antiga:

XanoScript VS Code Extension

A extensão antiga está deprecated. Para projetos novos, deve-se utilizar o **Language Server juntamente com o Xano CLI**.

O Language Server também não substitui o CLI. Ele ajuda o VS Code a compreender o código; o CLI é utilizado para sincronizar o código com o Xano.

7. Instalação do Xano CLI

Abra o terminal do VS Code:

Terminal → **New Terminal**

Execute:

```
npm install -g @xano/cli
```

Verifique:

```
xano --version
```

Se a versão for apresentada, a instalação foi concluída.

O Xano CLI é utilizado para operações como:

- autenticação;
- criação e gerenciamento de workspaces;
- pull do Xano para os arquivos locais;
- push dos arquivos locais para o Xano;
- trabalho com branches;
- trabalho com sandbox.

A documentação oficial do Xano CLI deve ser consultada caso os comandos sofram alterações.

[Documentação oficial do Xano CLI](#)

8. Autenticação do Xano CLI

Execute:

```
xano auth
```

O navegador será aberto para autenticação.

Após concluir o processo, confirme a conexão:

```
xano profile me
```

O objetivo é verificar se o CLI está associado à conta/instância correta.

Importante

O Xano CLI é o componente responsável pela conexão operacional com o workspace Xano.

Não confunda isso com o Xano Developer MCP.

O Developer MCP não requer token ou autenticação com o Xano. Ele funciona localmente e fornece documentação e validação. A conexão com o workspace é responsabilidade do CLI.

9. Criar o projeto local

Crie uma pasta para o projeto.

Por exemplo:

```
mkdir C:\Projetos\meu-projeto
```

```
cd C:\Projetos\meu-projeto
```

Uma estrutura recomendada é:

```
meu-projeto/
```

```
|
```

```
|— xano/
|
|— openspec/
|
|— .vscode/
|
|— ...
```

Abra a pasta no VS Code:

```
code .
```

10. Baixar o código do Xano

Se o workspace Xano já existir, utilize o Xano CLI para fazer o primeiro pull.

Por exemplo:

```
xano workspace pull -d ./xano
```

O resultado será uma representação local do workspace Xano em arquivos XanoScript.

Esses arquivos poderão ser editados no VS Code e versionados pelo Git.

Consulte sempre a documentação atual do CLI para confirmar a sintaxe dos comandos:

[Xano CLI — Push/Pull](#)

11. Inicialização do Git

Na raiz do projeto:

`git init`

Verifique:

`git status`

Adicione os arquivos:

`git add .`

Faça o primeiro commit:

`git commit -m "Initial project"`

O Git permitirá acompanhar todas as alterações realizadas nos arquivos XanoScript.

12. Criar o repositório no GitHub

Caso a disciplina utilize GitHub, crie um novo repositório na sua conta.

Depois associe o repositório remoto:

`git remote add origin https://github.com/SEU_USUARIO/SEU_REPOSITORIO.git`

Defina a branch principal:

`git branch -M main`

Envie o projeto:

`git push -u origin main`

A partir desse momento:

Computador

|

| `git push`



GitHub

O GitHub será utilizado principalmente para controle de versão, colaboração e histórico.

13. Instalação do GitHub Copilot

O GitHub Copilot pode ser utilizado diretamente no VS Code.

Consulte:

[Configuração oficial do GitHub Copilot no VS Code](#)

No VS Code:

1. Faça login com sua conta GitHub;
2. habilite os recursos de IA;
3. abra o Copilot Chat;
4. utilize o modo **Agent** quando quiser que a IA trabalhe com ferramentas e arquivos do projeto.

O VS Code atualmente integra suporte a agentes e MCP diretamente ao ambiente de desenvolvimento.

14. O que é MCP?

MCP (Model Context Protocol) é um padrão que permite conectar agentes de IA a ferramentas e fontes externas.

No nosso caso:

GitHub Copilot

|

| MCP



Xano Developer MCP

|

├─ documentação XanoScript

├─ validação XanoScript

├─ documentação CLI

├─ documentação Meta API

└─ documentação Run API

O MCP não é o agente de IA.

Ele é uma ponte entre o agente e determinadas ferramentas/informações.

15. Xano Developer MCP

O Xano Developer MCP é o componente que fornece ao agente conhecimento especializado sobre Xano.

Ele permite ao agente:

- consultar documentação XanoScript;
- consultar referências de sintaxe;
- validar XanoScript;
- consultar documentação do Xano CLI;
- consultar documentação da Meta API;
- consultar documentação da Run API.

O Xano Developer MCP foi desenvolvido especificamente para desenvolvimento com XanoScript.

A instalação normalmente utiliza:

`npx -y @xano/developer-mcp`

O MCP funciona como um servidor local utilizando comunicação **stdio**.

Não é necessário iniciar manualmente esse comando toda vez. Quando configurado no VS Code ou no agente, o cliente MCP gerencia o processo.

16. Configuração do Xano Developer MCP no VS Code + GitHub Copilot

No projeto, crie:

```
.vscode/  
└─ mcp.json
```

O arquivo pode conter:

```
{  
  
  "servers": {  
  
    "xano": {  
  
      "command": "npx",  
  
      "args": [  
  
        "-y",  
  
        "@xano/developer-mcp"  
  
      ]  
  
    }  
  
  }  
}
```

O VS Code suporta configurações MCP em `.vscode/mcp.json`. Também é possível utilizar a configuração global do usuário.

Depois:

1. Abra a Command Palette:
`Ctrl + Shift + P`
2. Execute:
MCP: List Servers
3. Localize `xano`;
4. escolha **Start**.

O VS Code deverá iniciar o servidor e descobrir suas ferramentas.

Uma execução bem-sucedida apresenta mensagens semelhantes a:

Starting server xano

Connection state: Running

Xano Developer MCP server running on stdio

Discovered ... tools

A quantidade de ferramentas pode mudar conforme a versão do Developer MCP. Consulte a documentação atual para verificar a lista vigente.

17. Testar o Xano Developer MCP no Copilot

Abra o GitHub Copilot em modo Agent.

Faça uma pergunta que exija conhecimento específico do XanoScript, por exemplo:

Consulte a documentação do XanoScript utilizando o Xano Developer MCP e explique como fazer uma query em uma tabela.

Outro teste:

Utilize o Xano Developer MCP para validar este código XanoScript e identifique possíveis erros.

O objetivo é verificar que o Copilot consegue utilizar as ferramentas disponibilizadas pelo servidor MCP.

No VS Code, o MCP pode ser gerenciado por **MCP: List Servers**, inclusive para iniciar, parar, reiniciar e visualizar os logs.

18. Configuração do Xano Developer MCP para Gemini CLI

O Xano Developer MCP também é compatível com o **Gemini CLI**. O Xano documenta o Gemini CLI como um dos clientes suportados.

Primeiro, instale/configure o Gemini CLI conforme a documentação oficial do Google/Gemini.

Depois, o MCP pode ser adicionado utilizando:

```
gemini mcp add xano npx -y @xano/developer-mcp
```

O Gemini CLI permite configurar servidores MCP por projeto ou globalmente. A configuração de projeto fica em:

```
.gemini/settings.json
```

e a configuração do usuário fica em:

```
~/.gemini/settings.json
```

A documentação atual do Gemini CLI confirma que servidores **studio** podem ser definidos usando **command** e **args**.

Também é possível configurar manualmente:

```
{  
  
  "mcpServers": {  
  
    "xano": {
```

```
"command": "npx",  
  
"args": [  
  
  "-y",  
  
  "@xano/developer-mcp"  
  
]  
  
}  
  
}  
  
}
```

Depois verifique:

```
gemini mcp list
```

Dentro de uma sessão do Gemini CLI também é possível utilizar:

```
/mcp list
```

O servidor deverá aparecer como conectado.

[Documentação oficial do Gemini CLI — MCP](#)

19. GitHub Copilot e Gemini podem usar o mesmo projeto

É possível ter:

```
meu-projeto/
```

```
|
```

```
|— .vscode/
```

```
|   └─ mcp.json
|
|─ .gemini/
|   └─ settings.json
|
|─ xano/
|
|─ openspec/
|
└─ .git/
```

Assim, o mesmo projeto pode ser utilizado com diferentes agentes.

Por exemplo:

VS Code

```
|
|─ GitHub Copilot
|   |
|   └─ Xano Developer MCP
|
└─ Gemini CLI
    |
    └─ Xano Developer MCP
```

O MCP é justamente um protocolo padronizado para permitir que diferentes clientes utilizem servidores de ferramentas.

20. Xano Developer MCP × Xano MCP Server

É fundamental não confundir os dois.

Xano Developer MCP

É o utilizado neste manual.

Ele:

- fornece documentação;
- valida XanoScript;
- auxilia o agente de IA;
- funciona localmente;
- não precisa de token Xano;
- não administra diretamente o workspace.

Xano MCP Server

É outro recurso do Xano.

Ele permite que um agente de IA interaja diretamente com recursos do workspace Xano.

Portanto, possui implicações de segurança maiores.

Para uma atividade acadêmica, recomenda-se começar utilizando:

Agente IA



Developer MCP



escrever / consultar / validar

↓
Git

↓

Xano CLI

↓

Sandbox Xano

em vez de conceder imediatamente ao agente acesso direto para alterar o workspace.

O próprio Xano diferencia explicitamente o Developer MCP do gerenciamento da instância.

21. Fluxo de desenvolvimento recomendado

Depois de toda a instalação, o fluxo recomendado é:

1. Definir requisito

↓

2. OpenSpec

↓

3. Agente de IA

↓

4. Developer MCP

↓

5. Criar/modificar XanoScript

↓

6. Language Server



7. Validar



8. git diff



9. git commit



10. Xano CLI



11. Promoção para o ambiente final

22. Pull e Push

Antes de começar a trabalhar, se houver possibilidade de alterações feitas diretamente no Xano:

```
xano workspace pull -d ./xano
```

Depois de modificar os arquivos:

```
git status
```

e:

```
git diff
```

Faça o commit:

```
git add .
```

```
git commit -m "Descrição da alteração"
```

Antes de enviar alterações ao Xano, recomenda-se utilizar primeiro um dry-run quando disponível:

```
xano workspace push -d ./xano --dry-run
```

Somente depois de revisar as alterações deve-se realizar o push.

Consulte a documentação atual do Xano CLI para os comandos e opções disponíveis.

23. O que cada ferramenta faz

Ferramenta	Função
VS Code	Editor
XanoScript Language Server	Suporte ao XanoScript
Git	Controle de versão local
GitHub	Repositório remoto
Xano CLI	Sincronização com Xano
Xano Developer MCP	Conhecimento e validação para IA
GitHub Copilot	Agente de IA
Gemini CLI	Agente de IA alternativo
OpenSpec	Especificação do desenvolvimento

24. O que não deve ser confundido

Language Server $\neq$ MCP

O Language Server ajuda o **editor**.

O MCP ajuda o **agente de IA**.

MCP ≠ Xano CLI

O MCP fornece ferramentas/contexto para a IA.

O CLI realiza operações de sincronização com o Xano.

Git ≠ Xano

Git controla o código e seu histórico.

Xano executa o backend.

Developer MCP ≠ Xano MCP Server

O Developer MCP é voltado principalmente para documentação e validação.

O Xano MCP Server pode fornecer acesso operacional ao workspace e, portanto, exige cuidados adicionais.

25. Segurança

MCPs são ferramentas executáveis e devem ser tratados com os mesmos cuidados que qualquer software de terceiros.

Antes de instalar um MCP:

1. verifique o fornecedor;
2. verifique o pacote;
3. consulte a documentação oficial;
4. não coloque tokens diretamente em arquivos versionados;
5. não conceda permissões desnecessárias;
6. revise as ferramentas disponíveis;
7. evite permitir operações destrutivas sem confirmação.

O VS Code alerta explicitamente que servidores MCP locais podem executar código arbitrário na máquina e recomenda adicionar apenas servidores de fontes confiáveis.

No caso do **Developer MCP do Xano**, a documentação oficial informa que ele funciona localmente e não necessita de credenciais do Xano.

26. Não dependa exclusivamente do conhecimento da IA

Um princípio importante para o desenvolvimento assistido por IA é:

A IA não deve ser considerada a documentação oficial da ferramenta.

Xano, XanoScript, VS Code, Git, Gemini, GitHub Copilot e outras ferramentas podem sofrer alterações.

Quando houver dúvida sobre:

- sintaxe XanoScript;
- comandos do Xano CLI;
- configuração do MCP;
- comportamento do Xano;
- configuração do GitHub Copilot;
- configuração do Gemini;
- versões de ferramentas;

consulte primeiro a documentação oficial correspondente.

O Xano Developer MCP é especialmente útil porque permite ao agente consultar documentação específica do XanoScript, CLI e APIs.

Para ferramentas que não possuem documentação incorporada ao MCP, peça explicitamente à IA para consultar a documentação oficial atual ou faça você mesmo a consulta.

Exemplo de instrução:

Antes de implementar, consulte a documentação oficial atual do XanoScript e confirme se essa sintaxe continua válida.

Ou:

Não presuma a sintaxe. Consulte a documentação oficial da versão atual da ferramenta antes de responder.

27. Procedimento de verificação final

Ao concluir a instalação, o aluno deverá conseguir executar:

```
node --version
```

```
npm --version
```

```
git --version
```

```
xano --version
```

Depois:

```
xano profile me
```

O VS Code deverá reconhecer arquivos:

```
*.xs
```

O MCP deverá aparecer em:

```
Ctrl + Shift + P
```

```
→ MCP: List Servers
```

O servidor **xano** deverá iniciar sem erros e apresentar suas ferramentas.

No GitHub Copilot, o aluno deverá conseguir solicitar uma consulta específica ao Xano Developer MCP.

No Gemini CLI:

```
gemini mcp list
```

deverá apresentar o servidor Xano como conectado.

29. Checklist de instalação

- ☐ Instalar VS Code
- ☐ Instalar Git
- ☐ Instalar Node.js LTS
- ☐ Criar/acessar conta Xano
- ☐ Instalar XanoScript Language Server
- ☐ Instalar Xano CLI
- ☐ Executar `xano auth`
- ☐ Executar `xano profile me`
- ☐ Criar projeto local
- ☐ Executar `xano workspace pull`
- ☐ Inicializar Git
- ☐ Criar repositório GitHub
- ☐ Configurar GitHub Copilot ou outro agente
- ☐ Criar `.vscode/mcp.json`
- ☐ Configurar Xano Developer MCP
- ☐ Iniciar o servidor pelo VS Code
- ☐ Confirmar descoberta das ferramentas MCP
- ☐ Testar o MCP no agente de IA
- ☐ Opcionalmente configurar Gemini CLI
- ☐ Testar `git diff`
- ☐ Testar Xano CLI

30. Documentação oficial

Os alunos devem priorizar sempre as fontes oficiais.

Xano

[Xano Documentation](#)

Xano Developer MCP

[Xano Developer MCP — documentação oficial](#)

Xano CLI

[Xano CLI — documentação oficial](#)

Visual Studio Code

[VS Code](#)

VS Code — MCP

[VS Code — configuração de MCP](#)

Git

[Git](#)

Gemini CLI

[Gemini CLI — MCP](#)

GitHub Copilot

[GitHub Copilot no VS Code](#)

31. Princípio geral do ambiente

O objetivo deste ambiente não é simplesmente permitir que uma IA "escreva código".

O objetivo é estabelecer um processo controlado:

REQUISITO



ESPECIFICAÇÃO



IA



DOCUMENTAÇÃO / MCP



CÓDIGO



VALIDAÇÃO



GIT DIFF



COMMIT



DEPLOY

A IA é utilizada como **assistente de desenvolvimento**, mas o código continua sendo revisado, versionado e testado.